SPECTRAL

VOICE INTELLIGENCE

SVI

An AI-Based Speech Enhancement System Using Residual Attention U-Net Architecture

TECHNICAL PROJECT REPORT

PROJECT PROFILE	DETAILS
Project domain	Artificial Intelligence • Deep Learning • Digital Signal Processing
Core model	Residual Attention U-Net implemented with PyTorch
Input	Noisy speech audio
Output	Processed WAV audio (intended enhanced speech)
Development environment	Python on Windows 11; CUDA-enabled GPU used in the reported run
Report basis	Implementation details and training/inference results shared for the current prototype
Report date	3 November 2026

PROJECT STATUS: WORKING RESEARCH PROTOTYPE

Evaluation note: The current implementation can train a model, save checkpoints, and generate an output WAV file. However, the reported listening test did not demonstrate effective background-noise suppression. This report presents the achieved implementation honestly and identifies the validation work still required.

Report Contents

- 1. Abstract
- 2. Introduction and Problem Statement
- 3. Aim and Objectives
- 4. Scope of the Project
- 5. Technologies and Tools
- 6. System Architecture
- 7. Methodology
- 8. Model Architecture
- 9. Dataset Preparation
- 10. Training Configuration
- 11. Training Results and Interpretation
- 12. Inference Workflow
- 13. Current Evaluation
- 14. Limitations and Risks
- 15. Future Enhancements
- 16. Conclusion
- 17. References and Technical Notes

1. Abstract

Spectral Voice Intelligence (SVI) is an artificial-intelligence project that explores the enhancement of speech recordings affected by background noise. The system is designed to learn a mapping from noisy speech representations to cleaner speech representations using supervised deep learning. Its current implementation uses Python, PyTorch, short-time Fourier transform (STFT) preprocessing, and a Residual Attention U-Net. An input waveform is converted into a log-magnitude spectrogram, normalized, processed by the neural network, and reconstructed into a WAV waveform using phase information from the noisy input. The project includes paired audio data handling, a training and validation pipeline, checkpoint saving, and an inference script. In the latest reported run, 20 paired recordings were detected; the best validation loss was 0.165630 at epoch 21, and early stopping occurred at epoch 31. Although the software pipeline runs, listening feedback indicates that effective noise suppression has not yet been established. The next development stage therefore prioritizes dataset auditing, signal-reconstruction checks, and objective evaluation.

2. Introduction and Problem Statement

Speech recordings made in homes, classrooms, offices, and outdoor environments may contain fans, traffic, people talking, wind, or other unwanted sounds. These sounds can reduce speech clarity and make recordings harder to understand. Traditional filters can help with selected noise conditions, but they may not work equally well for changing or overlapping sounds.

The problem addressed by SVI is to develop a learning-based system that takes a noisy speech recording and estimates a cleaner speech signal while retaining important speech information. The main technical challenge is to reduce unwanted noise without removing useful speech components or introducing distracting artifacts.

3. Aim and Objectives

Aim: To develop and evaluate a deep-learning-based speech enhancement prototype using a Residual Attention U-Net.

- Prepare paired noisy and clean speech recordings for supervised model training.
- Build an audio preprocessing pipeline for mono conversion, resampling, length adjustment, and spectrogram generation.
- Implement a Residual Attention U-Net in PyTorch for noisy-to-clean spectrogram mapping.
- Train and validate the model, monitor losses, and save the best and latest checkpoints.
- Create an inference workflow that accepts a noisy audio file and exports a processed WAV file.
- Evaluate output quality using controlled listening tests and objective speech-quality metrics.

4. Scope of the Project

- The current scope is single-channel (mono) speech processing at a target sample rate of 16 kHz.
- The model operates on fixed-length audio segments of four seconds in the reported configuration.
- The current learning target is a clean log-magnitude spectrogram estimated from the noisy input representation.
- The present reconstruction method combines the predicted magnitude with the noisy input phase.
- The current deliverable is a research prototype and command-line inference workflow; a finished real-time application is future work.

5. Technologies and Tools

Technology / Component	Role in SVI
Python	Main programming language for preprocessing, training, and inference.
PyTorch	Neural-network construction, optimization, and model execution.
Torchaudio / audio utilities	Audio loading, resampling, and WAV file handling.
STFT and inverse STFT	Conversion between waveform audio and time-frequency representations.
Residual Attention U-Net	Learns a mapping from noisy to cleaner spectrogram representations.
CUDA-enabled GPU	Accelerates model training and inference when available.
Windows 11	Development and execution environment.

6. System Architecture

The current end-to-end processing sequence is:

01 Noisy speech audio input

02 Mono conversion and resampling to 16,000 Hz

03 Length adjustment to the configured four-second segment

04 STFT conversion and log-magnitude calculation

05 Spectrogram normalization using input statistics

06 Residual Attention U-Net prediction

07 Denormalization of the predicted spectrogram

08 Combine predicted magnitude with noisy-input phase

09 Inverse STFT waveform reconstruction

10 Export processed WAV audio

The current network performs direct spectrogram regression. Its output layer predicts a clean normalized spectrogram rather than a sigmoid-based mask. During training, the clean target is normalized using statistics derived from the corresponding noisy spectrogram. During inference, the model output is denormalized using the input statistics before waveform reconstruction.

7. Methodology

7.1 Audio preprocessing

1. Load the audio file and convert it to mono where required.
2. Resample the waveform to 16 kHz so the model receives a consistent sampling rate.
3. Pad or trim the waveform to the configured segment length.
4. Calculate the STFT using a Hann window, FFT size of 512, hop length of 128, and window length of 512.
5. Calculate magnitude and log magnitude using a small epsilon for numerical stability.
6. Normalize the noisy representation and apply the corresponding noisy-input statistics to the clean target.

7.2 Supervised learning

Training uses paired examples: a noisy recording and its corresponding clean reference. The network receives the noisy spectrogram and is trained to predict the clean spectrogram. Correct pairing and time alignment are essential. If a noisy recording does not correspond to its clean reference, the model receives an incorrect learning target.

7.3 Loss function and optimization

The reported training configuration uses a weighted combination of L1 and mean squared error (MSE):

$$\text{Total Loss} = 0.8 \times \text{L1 Loss} + 0.2 \times \text{MSE Loss}$$

L1 loss measures the average absolute difference between predicted and target spectrogram values. MSE penalizes larger squared differences more strongly. The reported optimizer is AdamW with an initial learning rate of 0.0001 and weight decay of 0.00001. Gradient clipping is configured at 1.0.

8. Model Architecture

The core model is a Residual Attention U-Net. A U-Net-style encoder extracts features at progressively lower time-frequency resolutions. Residual blocks support feature learning, while attention modules help the network emphasize useful feature information. The decoder progressively restores resolution and combines decoder features with corresponding encoder features through skip connections.

Model component	Purpose
Encoder	Extracts hierarchical features from the input spectrogram.
Residual blocks	Support deeper feature learning and information flow.
Attention modules	Reweight feature information to focus on relevant patterns.
Bottleneck	Processes compressed high-level representations.
Decoder	Reconstructs a spectrogram at the original feature resolution.
Output layer	Predicts the clean normalized spectrogram directly.

The latest shared configuration uses a base channel count of 16 and two bottleneck blocks. The model has one input channel and one output channel for the mono spectrogram representation.

9. Dataset Preparation

The project uses two corresponding folders: dataset/ noisy_speech and dataset/ clean_speech. Each noisy file should have a clean reference with the same filename and matching speech content. The latest training run detected 20 valid pairs, divided into 16 training samples and 4 validation samples.

Important dataset issue: An earlier project check reported approximately 1,217 pairs, while the latest training run detected only 20. This discrepancy must be investigated before drawing conclusions about model performance. The larger dataset should not be assumed to be valid until filenames, audio content, sample rates, and alignment have been checked.

10. Training Configuration

Parameter	Latest reported setting
Detected paired files	20
Training / validation split	16 / 4
Sample rate	16,000 Hz
Audio segment length	4 seconds
FFT size (N_FFT)	512
Hop length	128
Window length	512
Batch size	1
Optimizer	AdamW
Initial learning rate	0.0001
Weight decay	0.00001
Maximum epochs	100
Early-stopping patience	10 epochs
Loss function	0.8 × L1 + 0.2 × MSE
Execution device	CUDA GPU

11. Training Results and Interpretation

Metric	Reported result
Training stopped	Epoch 31 of 100
Best checkpoint epoch	Epoch 21
Best validation loss	0.165630
Training loss at epoch 31	0.155725
Validation loss at epoch 31	0.179311
Stopping reason	10 consecutive epochs without validation improvement
Saved checkpoints	best.pt and latest.pt

Early stopping ended training because validation loss did not improve for the configured patience period. The best checkpoint represents the model saved at epoch 21, not the final epoch 31. The lower training loss at the end does not by itself prove better performance on unseen audio.

Loss values are useful for monitoring optimization, but they do not directly establish how much background noise was removed or whether speech quality improved. Audio comparisons and independent test data are required.

12. Inference Workflow

7. Select a noisy speech WAV or supported audio file.
8. Load the trained checkpoint and reconstruct the model using the matching architecture.
9. Apply the same sample-rate, length, STFT, and normalization steps used during training.
10. Run the model to predict the clean normalized spectrogram.
11. Denormalize the prediction using the noisy input statistics.
12. Combine the predicted magnitude with the noisy phase and apply inverse STFT.
13. Save the resulting waveform in the outputs folder as an enhanced/processed WAV file.

13. Current Evaluation

The reported inference test produced an output in which the background noise sounded louder while the voice remained similar. This is an important negative result: it means that effective noise reduction has not yet been demonstrated. The project should therefore be described as a functioning training and inference prototype, not as a validated speech-enhancement solution.

Evaluation area	Current status
Audio loading and preprocessing	Implemented in the current pipeline
Model training	Completed for the latest small dataset run
Checkpoint saving and loading	Implemented
WAV inference output	Generated
Audible background-noise reduction	Not demonstrated in the reported listening test
Independent test-set evaluation	Still required
Objective quality metrics	Still required

14. Limitations and Risks

- The latest run used only 20 paired files, which is insufficient to establish robust generalization.
- The difference between the earlier reported dataset count and the latest detected count remains unresolved.
- Incorrect or misaligned noisy-clean pairs can prevent the model from learning the intended mapping.
- The current reconstruction reuses noisy-input phase; this may limit quality, but it has not been proven to be the cause of the reported issue.
- A falling training loss does not guarantee perceptually better speech.
- The current report does not contain verified PESQ, STOI, or SI-SDR improvement measurements.
- Performance across different languages, speakers, microphones, and noise environments has not yet been established.

15. Future Enhancements and Development Plan

Priority	Recommended work	Expected benefit
1 - Critical	Audit the dataset and restore valid paired recordings; verify matching filenames and alignment.	Reliable training targets and a trustworthy dataset count.
2 - Critical	Create a held-out test set that is never used for training or model selection.	More honest measurement of generalization.
3 - Critical	Run a reconstruction control test: compare the original noisy audio with audio reconstructed from its own STFT magnitude and phase.	Check whether preprocessing/reconstruction introduces artifacts.
4 - High	Inspect noisy, clean-target, and predicted spectrograms using consistent display scales.	Identify prediction range or learning problems.
5 - High	Evaluate with SI-SDR improvement and, where suitable tools are available, STOI and PESQ; conduct loudness-matched listening tests.	Quantitative and perceptual evidence of enhancement.
6 - High	Increase speaker, recording, and noise diversity; use carefully aligned crops and suitable augmentation.	Better robustness across real-world conditions.
7 - Later	Compare phase-aware or waveform-domain methods after the baseline pipeline is validated.	Potential improvement in reconstructed audio quality.
8 - Later	Develop a user interface for importing audio, previewing results, and exporting enhanced recordings.	A more accessible application experience.

A multilingual version is a possible future direction. Speech enhancement is primarily an acoustic task rather than a language-translation task, but performance across languages must be tested rather than assumed. A multilingual dataset should include varied speakers and recording conditions, with separate evaluation results for each language.

16. Conclusion

Spectral Voice Intelligence establishes a practical foundation for an AI-based speech-enhancement system. The current prototype includes paired-audio loading, STFT-based preprocessing, a PyTorch Residual Attention U-Net, model training, checkpoint management, and WAV inference. The latest reported run reached a best validation loss of 0.165630 at epoch 21 and stopped at epoch 31 through early stopping.

However, the listening test did not demonstrate effective background-noise suppression. The next milestone should therefore focus on verifying the dataset, checking the signal-reconstruction path, inspecting model predictions, and evaluating on unseen recordings with objective metrics and controlled listening tests. These steps are necessary to move SVI from a working deep-learning prototype toward a dependable speech enhancement application.

17. References and Technical Notes

- PyTorch documentation: https://pytorch.org/docs/
- Torchaudio documentation: https://pytorch.org/audio/stable/
- Short-Time Fourier Transform (STFT): a time-frequency analysis method used to represent how signal frequency content changes over time.
- U-Net: an encoder-decoder neural-network design with skip connections, adapted here for spectrogram regression.
- All project-specific metrics and configuration values in this report are taken from the latest implementation details and training logs shared for this project.

END OF REPORT